

Agenda

Introduction

- Trung Bui
- Most AI coding knowledge is from self-learning and curiosity
- Instead of sharing, I hope this would be a learning opportunity

The State Of AI Coding

AI App Builder

- The first time ever, software development is available for non-programmer
 - Everyone can develop their own software
 - Primary target non-programmer, vibe coding in purest form
 - Example
 - AI App Builder: [loveable](#), [v0 by Vercel](#), traditional website builder (Wix, Wordpress with AI addon), LLM provider chat interface

Vscode fork/ Vscode Extension With AI

- Battery included AI coding assistant tool with familiar user friendly UI/UX (vscode)
- Easy to get started, powerful, popular community
- Dislike due to historic personal experience, making vscode experience degrade, not ready to give up vscode yet
- Black box, Feature paywall, try to self their in-house subscription
- Example: [cursor](#), [Vscode Copilot](#), [Cline](#), [Kilo Code](#)

TUI Coding Assistant

- An CLI terminal program coding AI assistant
- The strongest point would be the ability to add non-interactive workflow (run inside CI/CD, act as an MCP server for other tool to talk to)
- Mostly opensource, extensibility is usually first class feature
- Also another way for LLM provider to sell their subscription

- Example: [Claude Code](#), [OpenCode](#), [Gemini CLI](#),

Where Is OpenCode?

- The primary focus is still non-vibe code
- Free model, easier to try without worrying about subscription
- Not the first
 - Being the first is usually being bloated, especially in fast moving areas like AI (like ChatGPT, ClaudeCode)
 - OpenCode doesn't try to innovate new features, they usually copy other features
- Sustainable open source without forcing their subscription down user's throat
- The theme is good looking (personal preference)
- Buggy for trying to play catchup

AI coding is not about replacing developers. It's about amplifying their capabilities.

Expectation For AI Coding

- **Project Types**
 - Better for greenfield (from scratch) project than brownfield (existing/legacy project without documented patterns, etc...)
- **Tech Stack**
 - Support quality based on the amount and quality of training data (garbage in > garbage out)
 - The more public repo, the more popular the stack (language, framework, library), the better the LLM understands/generates the code
 - Compiled support is better than scripting (dynamic typing) language due to strict type system
 - => Best option: **TypeScript**

Primary Use Cases For AI Coding (personal experience)

Onboarding And Headstart

- Especially on the new repo (for some one-off consulting task, etc...)
- AI Coding acts as a fast to deploy personal assistant, knowledge base, code search engine

- Help me to understand the new codebase better and faster to be able to work on my expertise
- Search and understand codebase faster

Reduce Friction

- Small but time-consuming task
 - Doing thing in convention
 - git commit with ticket ID, scope, title, description, etc... (or worse emoji)
 - MR description with both business affection and technical detail
 - Write doc string, doc string is good, typing is good, but boring and frustrating
 - Analyze log
 - Find log with pattern
 - Visualize userflow with provided logs
 - Produce some meaningful insight (background process timeline, root cause detection)
 - Write unit test (especially if there is already a pattern in the codebase)

Implement 80%

- 80/20 rule
- expect LLM to generate copyable code
- AI can do 80% of the required code change with 20% time consumption compared to manual
 - The quality is always in question
 - The developer always need to do the remaining 20%
- But still a win because reduces the cognitive load of the developer

LLM Provider

Subscription Based (Coding Plan)

- Specific model provider: [GLM Coding Plan](#), Claude/GPT/Gemini subscription, Minimax, etc, ...
- Open source provider [Ollama Cloud](#), [OpenCode Go](#)

The LLM subscription plan is currently unstable, policy changes frequently without notice, new type of limit introduced, peak hour, session/weekly/monthly limit.

Create a habit of ignoring token usage, depending on strong model without consideration

Token Based

- Charge per million token
 - Specific model provider
 - Gateway provider [Vercel Gateway](#), [OpenRouter](#), Netlify, Cloudflare, etc...
 - Aware of token caching
 - optimize system prompt
 - understand/apply AI coding features more efficiently
-

- Prefer openrouter for personal preference
 - Active community (can be used as a reference which model is worth it)
 - Include free model
 - Some cool dashboard, analytics feature
 - First class support in most coding agent

OpenCode Walkthrough

Installation

- Follow installation [guide](#)

Config

- Always prioritize project-based config, config-as-code

```
1  <project_root>/opencode/
2  ├── opencode.jsonc          # Main configuration file (also reads config.json)
3  ├── tui.json               # Terminal UI appearance and keybinding
4  settings[reference:3]
5  |
6  ├── agents/                # Custom agent definitions (plural form)
7  [reference:4]
8  ├── commands/              # Custom slash commands[reference:5]
9  ├── plugins/                # Installed plugins (if using OpenCode's plugin
10 system)
11 |
12 ├── node_modules/           # Node.js dependencies for the
13 environment[reference:8]
14 ├── bun.lock                # Bun's lockfile for dependency
15 management[reference:9]
```

```

11 | └─ package.json           # Defines the Node.js dependencies for
    | OpenCode[reference:10]
12 | └─ .gitignore            # Ignore rules for the config directory

```

My Default Config

```

1  {
2    "$schema": "https://opencode.ai/config.json",
3    "agent": {
4      "build": {
5        "model": "openrouter/deepseek/deepseek-v4-flash"
6      },
7      "explore": {
8        "model": "openrouter/deepseek/deepseek-v4-flash"
9      },
10     "plan": {
11       "model": "openrouter/deepseek/deepseek-v4-pro"
12     }
13   },
14   "autoupdate": false,
15   "experimental": {
16     "batch_tool": true,
17   },
18   "model": "openrouter/deepseek/deepseek-v4-pro",
19   "permission": {
20     "codesearch": "deny",
21     "doom_loop": "deny",
22     "external_directory": "deny",
23     "lsp": "deny",
24     "todowrite": "deny",
25     "webfetch": "deny",
26     "websearch": "deny"
27   },
28   "share": "disabled",
29   "small_model": "openrouter/deepseek/deepseek-v4-flash",
30   "snapshot": false
31 }

```

- Disable some on-by-default feature `autoupdate` , `share` , `snapshot`
- Deny some permission/tool
 - `codesearch` , `websearch` , `webfetch` : turn off to reduce noises as well as call to third party service (exa)
 - `todowrite` , `doom_loop` : mostly for agentic automatic long running session, aka vibe code
 - `external_directory` : security, avoid `rm -rf ~/`
 - `lsp` : Catch LSP server right after agent writes code, sound nice on paper but in practice it just makes the model confuses and doom loop trying to fix some minor errors

- `batch_tool`: Prompt the LLM to dispatch multi tool call (latest model from all family are trained for this)
- Specify preferred model for default and agent

OpenCode Experiment Feature

Some features are still in experiment but too good to ignore

```
1 | OPENCODE_EXPERIMENTAL=true opencode
```

Better Plan Mode

```
1 | OPENCODE_EXPERIMENTAL_PLAN_MODE=true opencode
```

The default `plan` agent uses the same prompt as `build`, just with read-only permission => not a useful `plan` at all. Other agentic tools write plans to Markdown files, which lets you ask subagents to review and revise them, save them for future reference, or start new conversations for implementation work.

Plan > Review > Build

- Review is 2-step verification
 - suggest a different model family
 - LLM judges other LLM works brutally
- => Use LLM as judge improve the plan quality dramatically

Agents, MCPs, Rules, Tools, Skills

Feature	Tool / Example	Context Window Impact	Mitigations & Best Practices
Agents (Subagents)	Claude Code Task tool, Gemini CLI Subagents, OpenCode custom agents	Positive: Saves parent context by isolating verbose work in a fresh, independent context window . Only a summary or final result is returned to the main session. Negative: Context is lost between	* Scoped Context Passing: Explicitly pass critical information (like a git diff or relevant files) from the parent to the subagent to avoid re-discovery.

Feature	Tool / Example	Context Window Impact	Mitigations & Best Practices
		parent and child, leading to redundant work. This can increase overall token usage across sessions.	
MCPs (Model Context Protocol)	Connecting to GitHub, Jira, or Slack MCP servers	Negative: Massive context bloat at startup. Tool definitions for all connected servers are loaded into context before any conversation starts, often consuming 15-50% of the window. For example, 7 servers can add ~67k tokens .	<i>Tool Search (Lazy Loading):</i> Claude Code now dynamically discovers tools on-demand, reducing initial token cost by ~85%. Meta-MCPs: Rube MCP exposes only a few "meta-tools" that dynamically fetch the right sub-tool, keeping context clean. * "unclog" Audits: Use tools to scan and measure context bloat from all sources.
Rules (Custom Instructions)	Claude Code CLAUDE.md, Gemini CLI GEMINI.md, OpenCode rules	Negative: Constant baseline overhead. All defined rules and memory files are injected into the system prompt for every single turn. They can silently eat up a lot of tokens before you even type a message.	<i>Hierarchical & Scoped Files:</i> Place specific rules in relevant subdirectories so they only load in the right context. Regular Audits: Use tools like <code>uncLog</code> to identify and remove redundant or overly verbose rules.
Tools (Built-in & Custom)	Bash, Read, Write, Edit, Grep, Glob, Web Search, custom bash scripts	Mixed. While essential for work, their tool definitions contribute to constant context overhead. More importantly, the output of tool calls (e.g., a large file read or a verbose test run) is what causes sudden, massive context inflation and rot.	<i>Deferred Loading:</i> Claude Code now defers loading even built-in system tools behind a Tool Search mechanism to save initial context space. Context Editing: Surgically remove or replace large tool outputs from the context without starting a new session. * /compact: Actively summarize the conversation history to reclaim space.
Skills	Bundles of instructions, tools, and rules	Negative: Compound Bloat. A skill is a package that typically includes its own rules, agents, and MCP tools, which all consume context. This can silently add significant overhead.	<i>Audit Regularly:</i> Track which skills are actually being invoked and remove those that are inactive for extended periods. Treat as MCP/Tool Issue: Since skills combine other features, their individual parts can be mitigated using the same strategies for

Feature	Tool / Example	Context Window Impact	Mitigations & Best Practices
			MCPs (lazy loading) and Rules (scoping).

You have available skills below.
 <title><description>

Some opinions

- Most development MCP can be translated with skill + CLI [context7](#)
- Most latest LLM are trained to use skill
- Prioritize skill over MCP for static or one-shot workflow
 - skill
 - context7: user query => find library ID => forward user query with library ID
 - exa: user query => search result => repeat
 - MCP - interact with 3rd party API and next action solely depend on the API response
 - chrome devtool MCP
 - Figma MCP
 - Jira MCP
- Prioritize command over rule over agent
 - Command, a fancy term of the prompt shortcut
 - Can do useful stuff with [shell output](#)
 - Rule, standardize the behavior, especially useful when overwrite agent trained behavior for coding standard
 - Dangerous because of implicity
 - Eat up context
 - Agent, want a purpose focus fresh session (agent = session)
 - seach codebase
 - search library/framework doc
 - code without questioning

Some security concern

- Only use official MCP => MCP is the easiest way to get prompt injection
- If it can be a skill, be it a skill
- Be careful when skill contains script

- Install skill from popular registry to reduce risk [vercel](#), [context 7](#). Avoid installing skill from personal or awesome repos
- Published skills usually have multiple conditions to support a wide range of targets, you may only need to manually copy one small section from the skill prompt.

Methodology, Framework, Agent Team, Etc

| Like the concept, but not the implementation.

- Mostly created by and for vibe coder
- Eat token like crazy, produce low value per token/cost
- Spec driven development
 - [openspec](#)
 - [speckit](#)
 - Produce a unconsumable amount of markdown files => developer mostly ignore reviewing and accept anything LLM produce
- Agent Team
 - [oh-my-opencode](#), [getshitdone](#), [bmad](#) and everything alike
 - If want to look at [oh-my-opencode slim](#)

| The methodology still provides value, but not at the scale of one person usage or improving personal productivity.

The most important value can be applied immediately is copy/paste the prompt for agent/rule/command/etc....

Steering The Agent

- One of the benefit over not vibe coding is the ability to interrupt and direct the LLM to the correct path
 - Wrong path just leads to wasted time, wasted token/money
 - Remove the unreal expectation where the LLM can understand everything you want from first try
- Other tool like ClaudeCode already has this or Gemini CLI has this as experimental feature.
- OpenCode does not have this
- The best way is to violently interrupt the model, and continue from the last message

Stop Treating The LLM As A Data Processor, Treat It As A Code Generator

- The thing wastes your context the most may not be you or your rules, but the tool call and tool result
- Every LLM tool call dumps the raw result into context window
 - An MCP API list response => the whole list ~20KB
 - The log file from read tool to analyze => 50 KB
- Instead of tool call and tool result, let the LLM writes the code interact with the tool result and consume only the final result
 - Get whole list as tool result => LLM: Alright, let me check for the result and return the today tasks only

```
1  const response = await fetch();
2  const result = response.data.map(task => task.date === today);
3  return result
```

- The plugin [context mode](#)
- Reference articles
 - [Tanstack Code Mode](#)
 - [Cloudflare Code Mode](#)

Git Worktree

- The non-AI feature that really need to be mentioned, the foundation of every AI assistant parallel feature
- Learn this can help boost your productivity even without using AI
- Git branch works by updating the files in place in the same folder
- Git worktree works by having the source code in a dedicated folder
- Node need for `git stash` when switching between coding and reviewing
- Can do A/B implementation
- Integrate with AI will compound the benefits

Study Public Github

- Using 3rd party crawling service like [zread.ai](#)
- Or clone the repo locally using [repomix](#) then feed the AI the XML bundle file and ask it

- Use case:
 - Search through the codebase how the code behave internally
 - Hotfix some framework/library via monkey patch
 - Look for the agent, skill, prompt, etc to manually copy and paste
 - Just study

Minutes